第32章源码阅读

由正式源文件生成的静态阅读副本。返回正文 · 运行说明

src/main/java/org/example/jvminternals/ch32/ForeignJourney.java

```
package org.example.jvminternals.ch32;

import java.lang.foreign.Arena;
import java.lang.foreign.FunctionDescriptor;
import java.lang.foreign.Linker;
import java.lang.foreign.MemorySegment;
import java.lang.foreign.SymbolLookup;
import java.lang.foreign.ValueLayout;
import java.lang.invoke.MethodHandle;
import java.lang.invoke.MethodHandles;
import java.nio.file.Path;

/** 用同步回调使库、段、桩与Arena的有效期处于同一个词法范围。 */
public final class ForeignJourney {
    private ForeignJourney() {
    }

    /** 本地调用约定不允许Java异常任意跨越回调边界；本回调只做有限整数运算。 */
    public static int adjust(int value) {
        return value + 10;
    }

    /** MethodHandle精确调用声明Throwable；异常交给进程入口，不吞掉失败上下文。 */
    public static void main(String[] args) throws Throwable {
        MemorySegment heap = MemorySegment.ofArray(new int[]{2, 3, 5});
        System.out.printf("CH32 堆内段 bytes=%d native=%s%n", heap.byteSize(),
heap.isNative());
        MemorySegment.Scope scope;
        try (Arena arena = Arena.ofConfined()) {
            scope = arena.scope();
            MemorySegment data = arena.allocateFrom(ValueLayout.JAVA_INT, 2, 3, 5);
            System.out.printf("CH32 本地段 bytes=%d alive=%s owner=%s first=%d%n",
                    data.byteSize(), scope.isAlive(),
data.isAccessibleBy(Thread.currentThread()),
                    data.get(ValueLayout.JAVA_INT, 0));
            Linker linker = Linker.nativeLinker();
            SymbolLookup symbols = SymbolLookup.libraryLookup(
                    Path.of(System.getProperty("ch32.library")), arena);
```

```
            FunctionDescriptor callbackType = FunctionDescriptor.of(ValueLayout.JAVA_INT,
ValueLayout.JAVA_INT);
            MethodHandle callback = MethodHandles.lookup().findStatic(
                    ForeignJourney.class, "adjust", callbackType.toMethodType());
            MemorySegment stub = linker.upcallStub(callback, callbackType, arena);
            MethodHandle transform =
linker.downcallHandle(symbols.findOrThrow("ch32_transform"),
                    FunctionDescriptor.of(ValueLayout.JAVA_INT, ValueLayout.JAVA_INT,
ValueLayout.ADDRESS));
            int result = (int) transform.invokeExact(7, stub);
            System.out.printf("CH32 下行调用及同步上行回调 result=%d%n", result);
            // 本地函数不保留段或回调指针；返回后才允许关闭所属Arena。
        }
        System.out.printf("CH32 Arena关闭后 alive=%s；不再解引用本地段或调用桩%n",
scope.isAlive());
    }
}
```

src/main/java/org/example/jvminternals/ch32/NativeJourney.java

```
package org.example.jvminternals.ch32;

/** 以短本地调用显示引用、临界区和外部线程的责任边界。 */
public final class NativeJourney {
    private NativeJourney() {
    }

    private static native boolean acquire(Object value);
    private static native long sumCritical(int[] values);
    private static native int attachedCallback();
    private static native void release();

    /** 由已附加的原生线程调用；不能把调用者线程的JNIEnv转交给它。 */
    public static void onNativeThread() {
        System.out.printf("CH32 原生线程回调：%s virtual=%s%n",
                Thread.currentThread().getName(), Thread.currentThread().isVirtual());
    }

    /** 强全局引用在finally显式释放；弱引用不承担保持对象存活的责任。 */
    public static void main(String[] args) {
        System.load(System.getProperty("ch32.library"));
        System.out.println("CH32 本地库已加载");
        try {
            boolean held = acquire(new byte[16]);
```

```
            System.out.printf("CH32 跨调用引用已建立=%s%n", held);
            if (held) {
                System.out.printf("CH32 临界区求和=%d%n", sumCritical(new int[]{2, 3, 5}));
                System.out.printf("CH32 附加线程返回状态=%d%n", attachedCallback());
            }
        } finally {
            release();
            System.out.println("CH32 全局及弱全局引用已释放");
        }
    }
}
```

---

native/interop.c

```
#include <jni.h>
#include <pthread.h>
#include <stdint.h>
#include <stdio.h>

/* 单主线程教学调用；共享槽只在创建原生线程之前写入、join之后释放。 */
static JavaVM *vm;
static jobject strong_ref;
static jweak weak_ref;

JNIEXPORT jint JNICALL JNI_OnLoad(JavaVM *loaded_vm, void *reserved) {
    (void)reserved;
    vm = loaded_vm;
    return JNI_VERSION_1_8;
}

JNIEXPORT jboolean JNICALL Java_org_example_jvminternals_ch32_NativeJourney_acquire
  (JNIEnv *env, jclass type, jobject value) {
    (void)type;
    strong_ref = (*env)->NewGlobalRef(env, value);
    if (strong_ref == NULL) {
        return JNI_FALSE;
    }
    weak_ref = (*env)->NewWeakGlobalRef(env, value);
    if (weak_ref == NULL) {
        (*env)->DeleteGlobalRef(env, strong_ref);
        strong_ref = NULL;
        return JNI_FALSE;
    }
    return JNI_TRUE;
}
```

```
JNIEXPORT jlong JNICALL Java_org_example_jvminternals_ch32_NativeJourney_sumCritical
  (JNIEnv *env, jclass type, jintArray values) {
    (void)type;
    /* 先把弱引用提升为局部强引用；此例还保留全局强引用，不制造回收竞态。 */
    jobject local = (*env)->NewLocalRef(env, weak_ref);
    if (local == NULL) {
        return -1;
    }
    (*env)->DeleteLocalRef(env, local);
    jsize length = (*env)->GetArrayLength(env, values);
    jboolean copied = JNI_FALSE;
    jint *elements = (*env)->GetPrimitiveArrayCritical(env, values, &copied);
    if (elements == NULL) {
        return -1;
    }
    jlong total = 0;
    /* 临界区内只访问这段数组；不打印、不阻塞、不回调或调用其他JNI函数。 */
    for (jsize index = 0; index < length; index++) {
        total += elements[index];
    }
    (*env)->ReleasePrimitiveArrayCritical(env, values, elements, JNI_ABORT);
    printf("CH32 临界区已释放 isCopy=%s\n", copied == JNI_TRUE ? "true" : "false");
    fflush(stdout);
    return total;
}

struct CallbackContext {
    jclass type;
    jmethodID callback;
    int status;
};

/* 原生线程必须取得自己的JNIEnv，并在退出前分离；类型由调用方全局引用保持。 */
static void *run_callback(void *argument) {
    struct CallbackContext *context = argument;
    JNIEnv *env = NULL;
    JavaVMAttachArgs args = {JNI_VERSION_1_8, "ch32-attached", NULL};
    context->status = (*vm)->AttachCurrentThread(vm, (void **)&env, &args);
    if (context->status != JNI_OK) {
        return NULL;
    }
    (*env)->CallStaticVoidMethod(env, context->type, context->callback);
    if ((*env)->ExceptionCheck(env)) {
        /* 无Java调用栈的外部线程在本边界报告异常；不携带待处理异常继续JNI工作。 */
        (*env)->ExceptionDescribe(env);
        (*env)->ExceptionClear(env);
        context->status = JNI_ERR;
    }
    jint detached = (*vm)->DetachCurrentThread(vm);
    if (detached != JNI_OK) {
        context->status = detached;
```

```
    }
    return NULL;
}

JNIEXPORT jint JNICALL Java_org_example_jvminternals_ch32_NativeJourney_attachedCallback
  (JNIEnv *env, jclass type) {
    struct CallbackContext context = {NULL, NULL, JNI_ERR};
    context.callback = (*env)->GetStaticMethodID(env, type, "onNativeThread", "()V");
    if (context.callback == NULL) {
        return JNI_ERR;
    }
    context.type = (*env)->NewGlobalRef(env, type);
    if (context.type == NULL) {
        return JNI_ERR;
    }
    pthread_t worker;
    int created = pthread_create(&worker, NULL, run_callback, &context);
    if (created == 0) {
        /* 成功创建的线程必须join后才销毁栈上上下文和全局类引用。 */
        int joined = pthread_join(worker, NULL);
        if (joined != 0) {
            /* 此编程错误不能安全返回并留下悬空栈指针，终止当前进程。 */
            (*env)->FatalError(env, "CH32 cannot join owned native thread");
        }
    } else {
        fprintf(stderr, "CH32 创建本地线程失败：%d\n", created);
    }
    (*env)->DeleteGlobalRef(env, context.type);
    return context.status;
}

JNIEXPORT void JNICALL Java_org_example_jvminternals_ch32_NativeJourney_release
  (JNIEnv *env, jclass type) {
    (void)type;
    if (weak_ref != NULL) {
        (*env)->DeleteWeakGlobalRef(env, weak_ref);
        weak_ref = NULL;
    }
    if (strong_ref != NULL) {
        (*env)->DeleteGlobalRef(env, strong_ref);
        strong_ref = NULL;
    }
}

/* 固定int32_t契约；同步调用一次回调，不在返回后保留函数指针。 */
int32_t ch32_transform(int32_t value, int32_t (*callback)(int32_t)) {
    return callback(value) * 2;
}
```